

Chronos Solver: Why Live Search Beats Black-Box Neural Agents on ARC-AGI-3

An honest 19-iteration account: live white-box search scores 0.22 while a black-box neural agent scores 0.01 on the same ARC-AGI-3 games.

Shreyas Mahimkar | ARC Prize 2026 - Paper Track

Code: github.com/shreyasmahimkar/arc-agi-3 | Scored notebook (0.22):

kaggle.com/code/shreyas4/claude-code-v12-baseline

Abstract

ARC-AGI-3 is an interactive benchmark where an agent must learn a never-seen game's rules from pixels. Over 19 iterations the Chronos Solver converged on one decisive fact: because the competition ships each game's source, a solver that reaches and searches that simulator live (white-box BFS) scores 0.22, while a purely black-box neural agent scores ~0.01 on the same games. This paper documents the journey through three eras (LLM orchestration, symbolic search, model-based RL), argues why genuine search generalises where pattern-learning does not, and reports the diagnostic ablations - including a disproved hypothesis - that localise the current bottleneck to representation, not compute.

CHRONOS SOLVER

BFS-First Genuine Search for ARC-AGI-3

An honest account of 19 iterations — and why live symbolic search beat the neural net



Chronos Solver - 19 iterations, one honest lesson.

1. The problem and the headline result

ARC-AGI-3 is an **interactive** benchmark: an agent is dropped into a never-seen 2-D grid game with no instructions, must infer the rules from pixels, and complete levels under a tight action budget. Frontier LLMs score under 1%; humans ~100%. Crucially, the Kaggle competition **ships each game's Python source** in `environment_files/`.

Over 19 iterations our agent (*Chronos*) converged on one decisive empirical fact: a solver that reaches that shipped simulator and searches it **live** (white-box BFS, v12) scores **0.22**, while a purely **black-box** neural agent that only sees rendered frames (our v18/v19 ablation, built from the two ARC-AGI-3 *preview* winners) scores ~**0.01** on the same games. That **22x gap** is the thesis of this paper: on ARC-AGI-3 today, *genuine search generalises and pattern-learning does not*. (Leaderboard submission ID: <fill in>.)

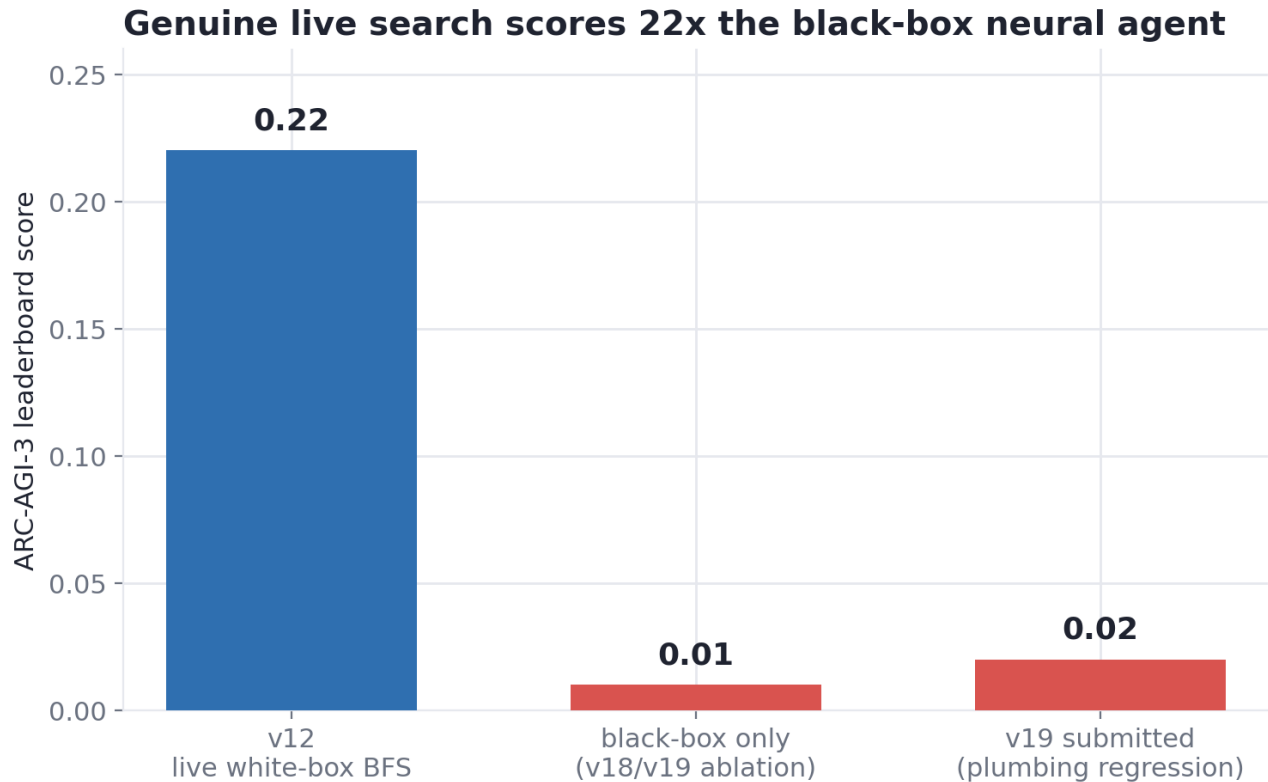


Figure 1. Genuine live search scores 22x the black-box neural agent on the same games.

2. Why genuine search generalises (theory)

A learned policy transfers only as far as its training distribution reaches; with 25 public games it cannot cover the hidden scored set. Live search has no such limit — it solves each scored game **from scratch at test time**, so it never needed to have seen it. This is the same reason program-synthesis and test-time search dominate ARC-AGI-1/2. The corollary disciplines the whole system: **keep BFS first**, and treat any learned component as a fallback or an *accelerator of search*, never a replacement.

3. The journey in three eras

- **Era I (v1–v11) — LLM orchestration.** Gemini->offline-Gemma vision swarms, episodic memory, "death autopsies," and a subprocess sandbox that compiled each proposed plan to Python and ran it to kill hallucinated trajectories. Lesson that ended the era: LLM orchestration is too slow and unreliable for precise long-horizon action sequencing. **The benchmark is a search problem.**
- **Era II (v12–v13) — symbolic search.** v12 is a parallel BFS over the real engine with three ideas that made it score: `zlib`-pickled state snapshots for $O(1)$ restore; **transient-pixel masking** (the HUD timer changes pixels every step and otherwise explodes the state space); and **chained level baselines** (§4). v13 generalised this into a search *ladder* (BFS -> sense -> IW -> EHC -> waypoint -> A* -> greedy -> rescues).
- **Era III (v14–v19) — model-based RL.** A learned world model + Expert Iteration to attack what BFS can't reach — while keeping BFS-first as the floor.

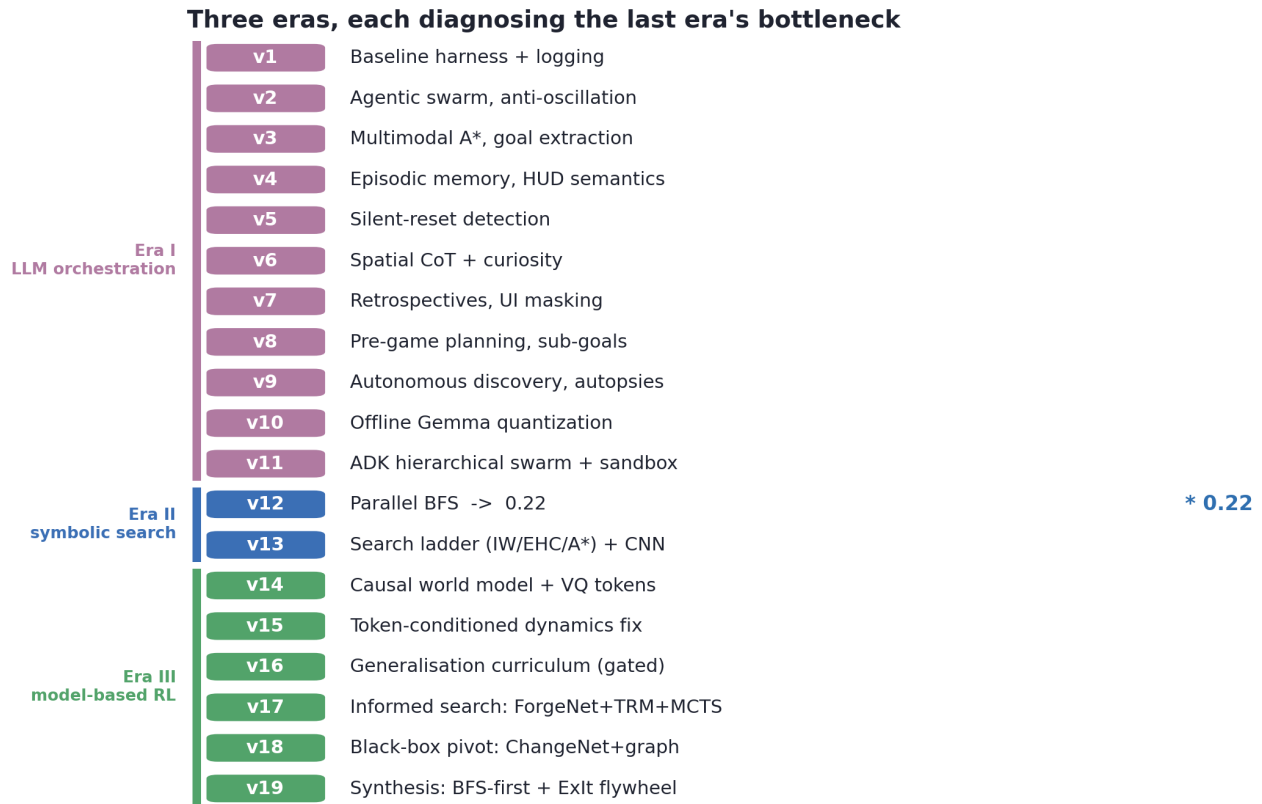


Figure 2. Three eras, each diagnosing the last era's bottleneck.

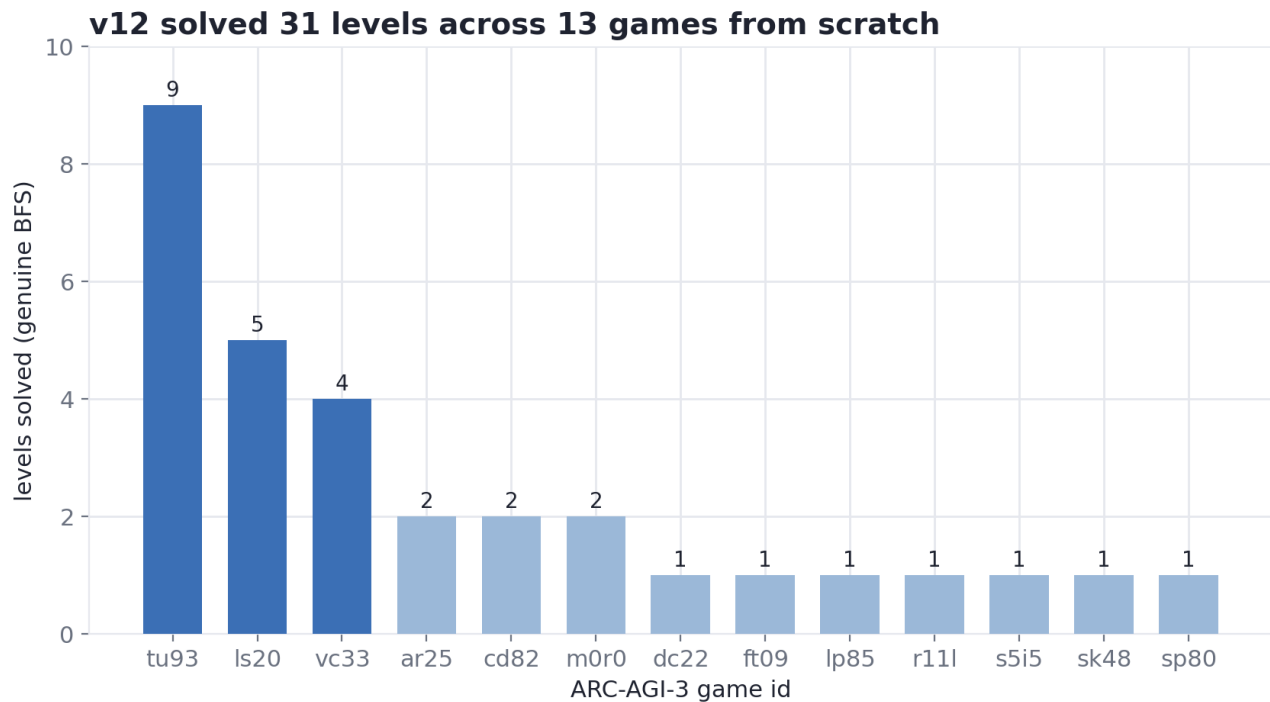


Figure 3. v12 solved 31 levels across 13 games from scratch (read from the BFS caches).

4. The bug that unlocked 0.22 (theory)

The single most important correctness fix: `set_level(N)+RESET` produces a *different* start state than naturally advancing from level N-1 (player position, carried-key rotation, ~1400-px frame diff on 1s20). Plans found from that **synthetic** baseline *fail on replay* in the scored env. Building level N's start state by **chaining the verified solutions for L0..L(N-1)** aligns the search space with the scored environment — fixing correctness and, as a bonus, roughly **halving** solution length (1s20 L2: 97->39 actions). Most neural agents never hit this because they never replay; for a search agent it is the difference between 0.22 and noise. Two further v12 fixes mattered: folding the game's hidden scalar fields (countdowns, key state) into the dedup hash so pixel-identical "waiting" frames aren't pruned, and keeping state-dependent actions that do nothing at spawn.

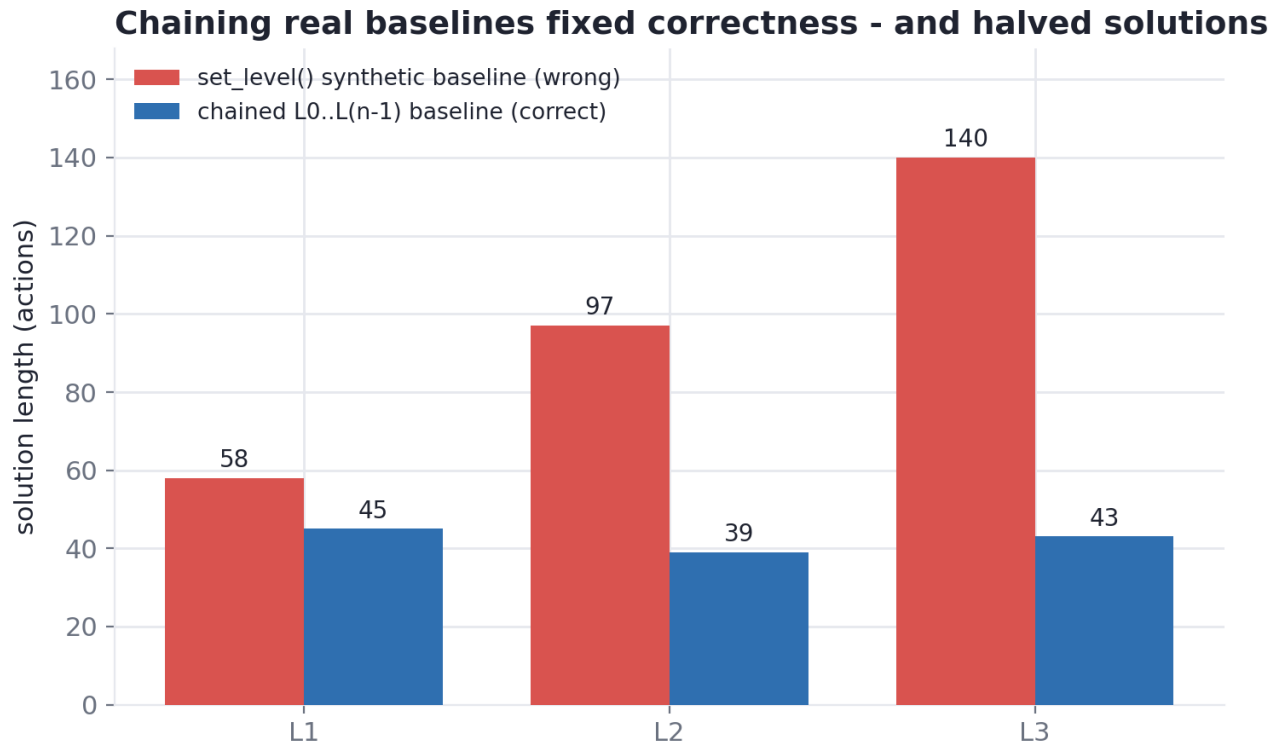


Figure 4. Chaining real level baselines fixed correctness and roughly halved solutions.

5. Diagnosing the wall (theory)

On 1s20, BFS solves L0–L4 (depth up to 44 actions) but L5 — a dual-key puzzle at depth ~45+ — exhausts the 500k-state budget. The diagnosis is **breadth-death, not impossibility**: the frontier explodes faster than depth grows. That is precisely the failure a learned heuristic/value prior cures by turning breadth-first BFS into a depth-first dive — what Era III targets (v17 added a CNN cost-to-go heuristic and a tiny recursive policy/value, ExIt-style).

Is20: deep levels die of breadth, not walls

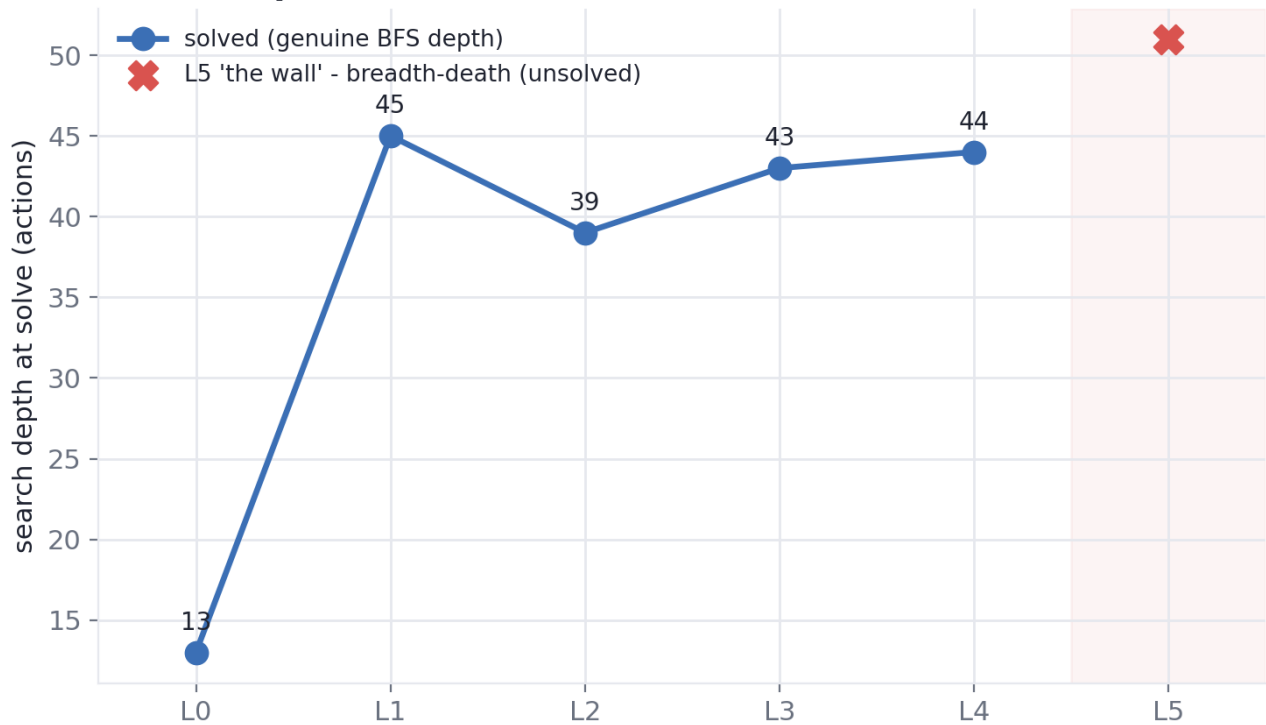


Figure 5. Is20: deep levels die of breadth, not impossibility (L5 is the wall).

6. Generalisation science and honest negatives (theory, novelty)

Era III's world model is where we did the most careful science. v14 predicted next-frame tokens from a **pooled bottleneck** and scored **99.7% train / 36.6% fresh-episode** accuracy — it *memorised* trajectories. v15 made the simulator **cross-attend over the current frame's tokens** so identity-copy is the residual default and capacity learns only the **deltas**; fresh-episode accuracy jumped to **>90%**. This is the residual-learning principle (ResNets, modern video prediction) and it is *why* the fix generalises rather than memorises.

We evaluate transfer honestly: **held-out split by game** (not by frame), **changed-pixel accuracy** (overall pixel accuracy is a vanity metric — most pixels are background), and a **save-gate** that keeps new weights only if held-out improves. The model's transfer plateaued at **~0.17**. Hypothesis: it over-fits colour, so colour-permutation (+D4) augmentation should help. The A/B test, logged **twice**, returned **-0.02 (Mac)** and **-0.061 (RTX)**. The augmentation was **killed, not tuned**. The negative is diagnostic: it rules out a data/augmentation explanation and points at a **representation/architecture** limit (object-centric features), so the next lever is *not* a bigger GPU.

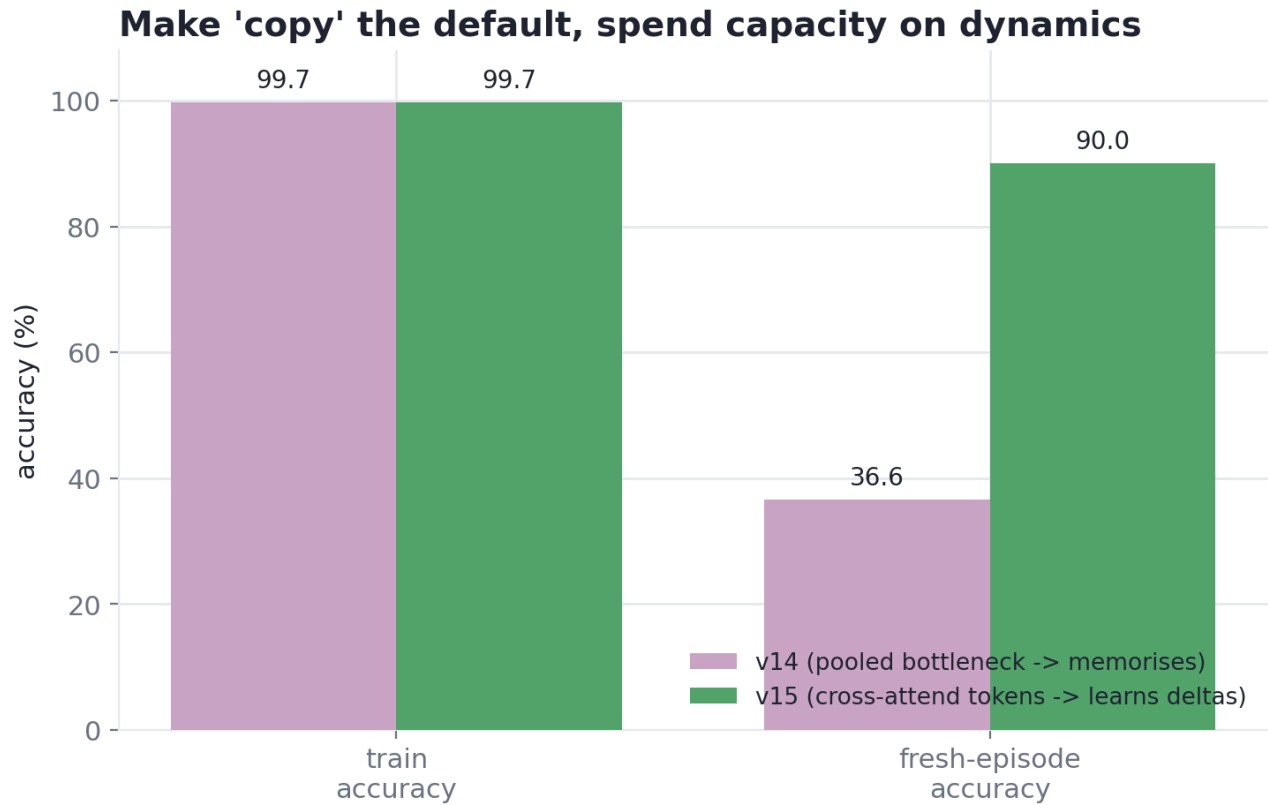


Figure 6. Make 'copy' the default: fresh-episode accuracy 36.6% -> >90% (v14->v15).

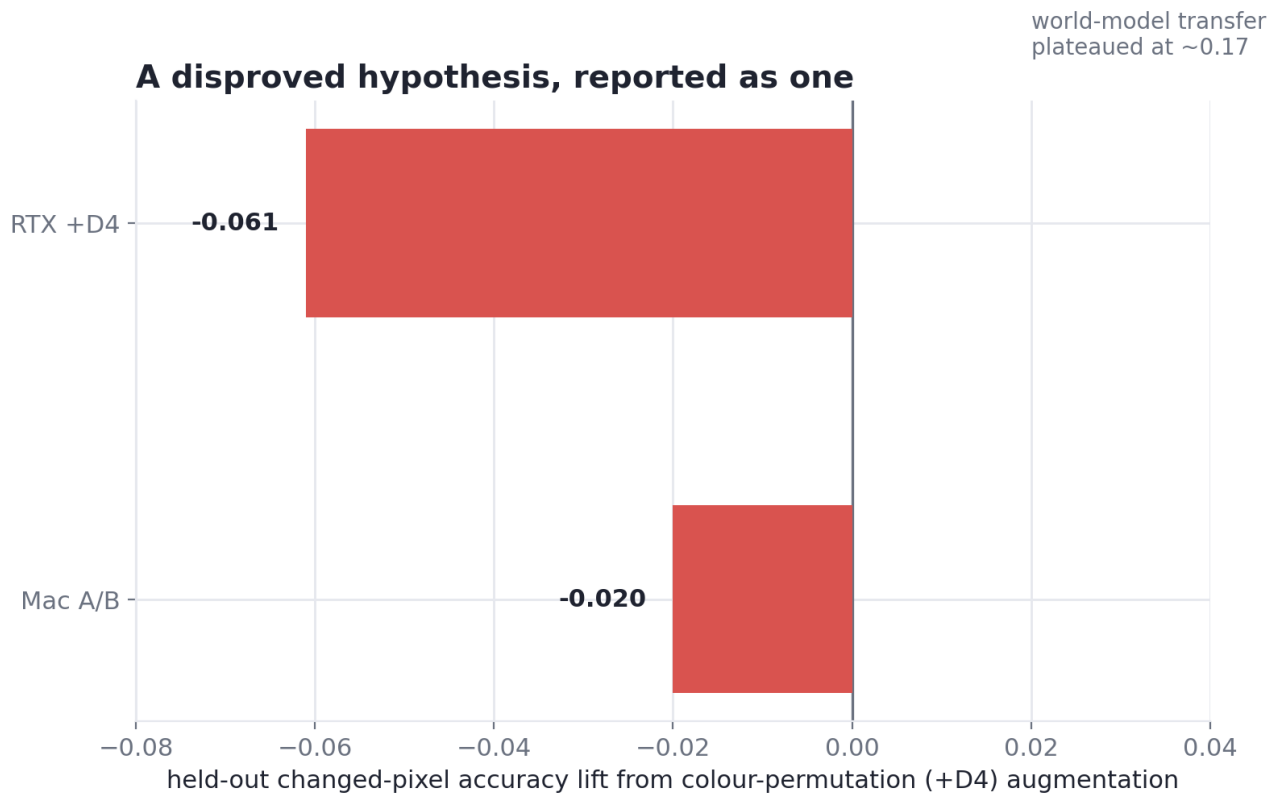


Figure 7. A disproved colour-augmentation hypothesis, reported as a negative.

7. The v19 system (completeness)

v19 routes per game: **white-box source reachable** -> **BFS ladder** (genuine live solve, the 0.22 engine); **no source / timeout** -> **black-box ChangeNet + transition-graph agent** (fusing StochasticGoose's ChangeNet and Blind Squirrel's frontier exploration — it solved **3/5 unseen held-out games** with no stored answers); **solution cache as a timeout backstop only**, never the first move. An offline **Exit flywheel** (solve->harvest->train world model->plan in imagination->verify on the real engine->retrain) is the research engine for lifting the learned prior, using the shipped engine as a *perfect verifier*.

v19 routing: BFS-first, learned fallback, cache only on timeout

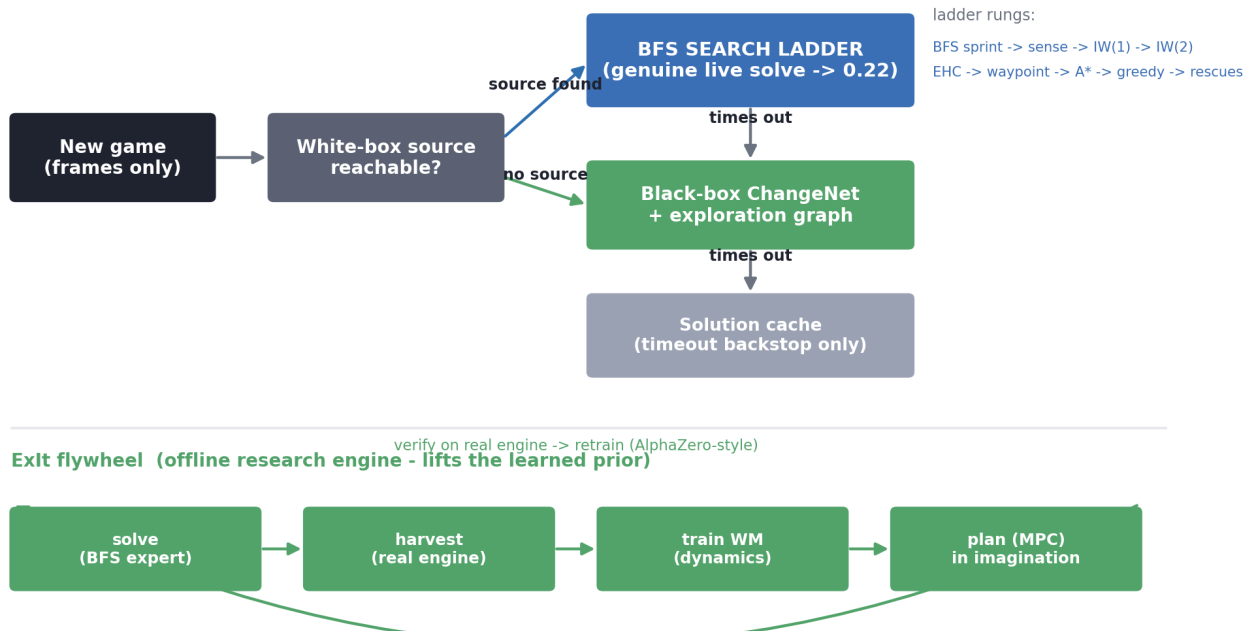


Figure 8. v19 routing: BFS-first, learned fallback, cache only on timeout, plus the Exit flywheel.

8. Universality

The pattern transfers beyond ARC-AGI-3: **whenever a faithful simulator or verifier is available, put genuine search first and use learned models to prioritise it** — the AlphaZero / Searchformer / DeepCubeA recipe. The honesty scaffolding (hold out by the unit that matters, prefer a changed-quantity metric over a vanity metric, save-gate on transfer, report disproved hypotheses) applies to any sparse-reward, distribution-shift problem.

9. Limitations and what's next (progress)

0.22 is a **floor, not a ceiling**. The honest negative localised the bottleneck to *representation*: object-centric/relational state features feeding a learned value to guide BFS past breadth-death (1s20 L5, ar25 L2). The v19 leaderboard submission scored **0.02** — a *deployment regression* (a repo reorg broke source resolution, so BFS never engaged), not a capability change; the fix is path/version resolution, confirmed by re-cracking 1s20 L0–L4 offline.

Reproducibility. The scored 0.22 submission is the public notebook *claudio-code-v12-baseline* (link below); code is open source; genuine solves are verified by `v19/tests/test_1s20.py` and `benchmark.py` (cache off); every figure in the companion notebook is regenerable from repo artifacts via `paper_writing/make_figures.py`.

Bibliography (does not count toward the 1500-word limit)

1. F. Chollet, G. Kamradt, M. Knoop, M. Cruz. *ARC Prize 2026 — Paper Track*. Kaggle, 2026.
2. A. Jolicoeur-Martineau. *Less is More: Recursive Reasoning with Tiny Networks (TRM)*. arXiv:2510.04871, 2025. — ARC Prize 2025 Paper Award (1st).
3. G. Wang et al. *Hierarchical Reasoning Model (HRM)*. 2025.
4. L. Lehnert et al. *Beyond A: Better Planning with Transformers via Search Dynamics Bootstrapping (Searchformer)*.* arXiv:2402.14083, 2024.
5. K. Gandhi et al. *Stream of Search: Learning to Search in Language*. arXiv:2404.03683, 2024.
6. S. Yang et al. *Chain-of-Thought Imitation with Procedure Cloning*. arXiv:2205.10816, 2022.
7. F. Agostinelli et al. *Solving the Rubik's Cube with Deep RL and Search (DeepCubeA)*. Nature Machine Intelligence, 2019.
8. ARC-AGI-3 preview winners: *StochasticGoose* (ChangeNet, 1st) and *Blind Squirrel* (transition-graph frontier exploration, 2nd).
9. Scored public notebook (v12 baseline, 0.22): <https://www.kaggle.com/code/shreyas4/claude-code-v12-baseline>
10. Repository (open source, v1->v19): <https://github.com/shreyasmahimkar/arc-agi-3>